



DB Copilot

数据库智能运维平台

PRESENTATION · LLM-NATIVE DB OPS



数据库智能运维平台

DB Copilot · 项目规划 / 架构设计 / 实施进展 / 任务分派

监控大盘 · 智能诊断 · 多 Agent 对话 · 统一数据底座

汇报框架



01

目标与范围

最终形态与建设目标
MVP 边界与北极星场景



02

总体架构

分层架构与技术选型
核心流程时序与数据流



03

关键模块

Agent 执行框架 / 异步任务闭环
工具注册表 / 卡片协议 / 前端进展



04

分工与验收

三个任务系列与排期
契约优先验收机制与风险

本页为全局导航：四部分依次对应「为什么做 → 怎么设计 → 关键如何实现 → 谁来做如何验收」

背景与痛点：存量割裂，能力待统一收口



存量系统各自割裂

告警系统、工单系统、DBaaS 平台独立建设，入口分散、体验不一，排障需在多系统间来回切换



外采诊断能力受限

已外采诊断 Agent 覆盖大部分国产数据库，API 可编程；但自带界面不满足需求，难以融入统一平台

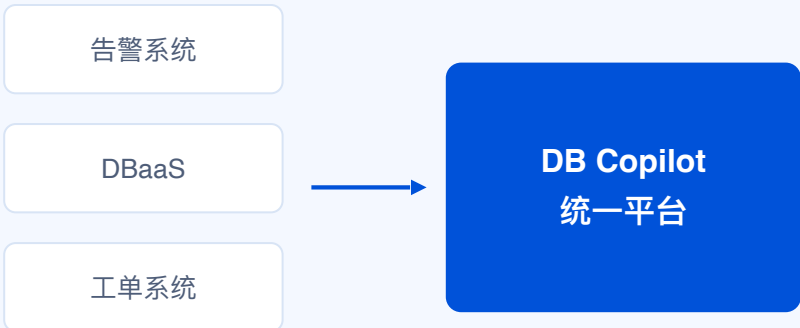


数据体系与 API 风格不一

指标、元数据、告警的数据模型与接口风格各异，新能力建设反复适配、重复投入

建设契机 · 渐进替换策略

Strangler Fig：新建统一平台，旧系统经防腐层集成、按模块分批吸收，最终按节奏下线



防腐层收口：旧系统变更只波及适配层

建设目标与最终形态：双范式智能运维平台

五大建设目标

1

综合型智能运维平台

覆盖多种数据库类型的统一入口

2

保留并重构传统能力

概览 / 集群 / 监控大盘 / 实例详情

3

重点建设下钻分析

概览→集群→实例→会话/慢SQL/锁

4

对话式智能运维

LLM 多 Agent：诊断 / 问数 / 知识

5

兼容旧体系渐进替换

适配旧数据体系与 API，支持分批吸收

双范式并列 · 双向流转



浏览式

监控大盘（Grafana 式编辑器）
集群 / 实例详情 · 下钻导航



对话式

Copilot 智能对话
AI 输出结构化卡片渲染

愿景能力（二期+ 设计占位已入架构）

选中页面元素 · 直接智能诊断

平台说明书知识库 · 回答附跳转 URL

一期北极星功能：数据库实例智能诊断

MVP 范围：3 个月 · 3 人 · 真实数据接入



一句话目标：以真实数据跑通五项架构核心假设 —— 统一查询协议 · 多 Agent 编排 · 卡片协议 · 异步任务闭环 · 防腐层集成



MVP 交付（做）

- 监控大盘：完整编辑器复刻（多大盘 / 双 Y 轴 / 阈值 / 事件标注）
- 智能对话：多会话 + 流式 + 5 种结构化卡片
- Agent 诊断：LangGraph ReAct + 8 个自建工具
- Agent 问数：四类白名单 NL2Metric
- 真实接入：公司 AI 平台 / 旧监控 / DBaaS / 告警
- 外采诊断：适配器规范就绪，契约到手即接入



明确不做（防范围蔓延）

- 鉴权 / SSO（权限桩，二期接 RBAC）
- 页面上下文注入 · 元素选中诊断（二期）
- 自治服务页 · 知识库问答（二期）
- L1 / L2 动作执行（Agent 只读提议）
- Redis / 消息队列 / 对象存储（PG 单库）
- MCP / CLI / Skills 插件生态（二期+）

北极星验收场景

实例变慢



大盘发现异常



chat 发起诊断



真实证据采集



异步深扫任务



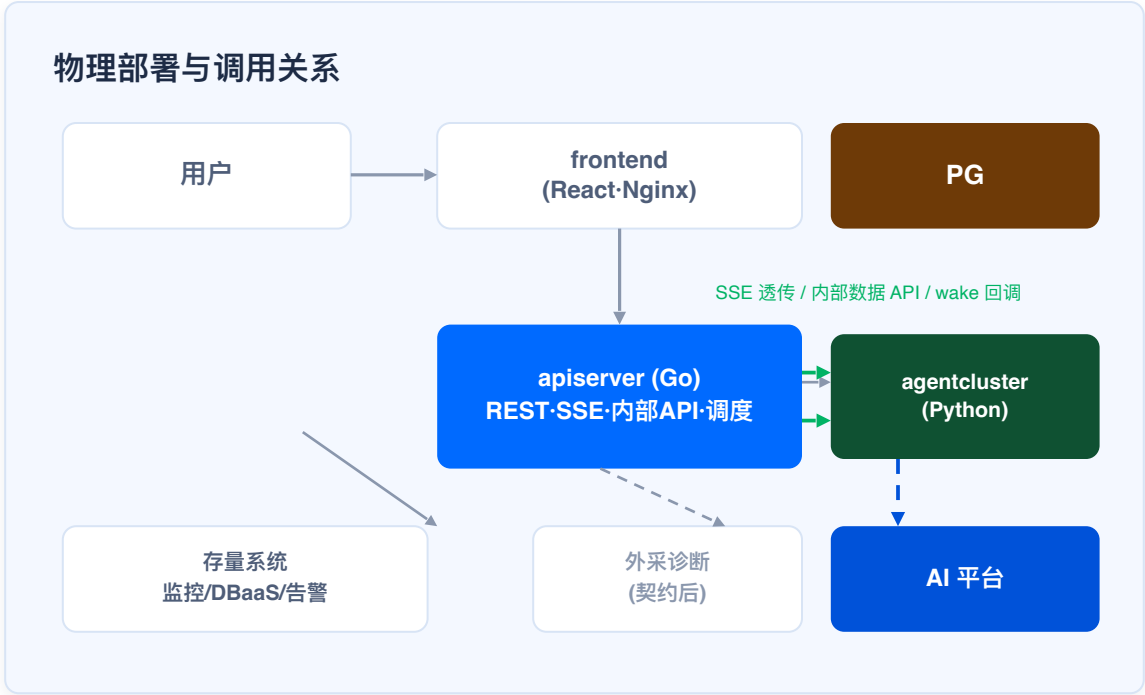
报告卡 + 追问

分层架构：逻辑分层 × 物理归属（React / Go / Python / PG）



技术选型与部署形态：Compose 四容器起步

层	选型	要点
前端	React 18 + TS + Vite	卡片渲染器 = 组件注册, ECharts
apiserver	Go	收口入口与存储, SSE 透传
Agent 集群	Python + LangGraph	四端口封装, 无状态多副本
LLM	公司统一 AI 平台	OpenAI 兼容 · tool calling · 流式
存储	PostgreSQL 单库	JSONB 承载卡片 / 轨迹; 不引入中间件
部署	Docker Compose	生产 K8s 后置



Go ↔ Python 边界三原则（硬约束）

入口收口

前端一切请求（含 chat SSE）只打 apiserver

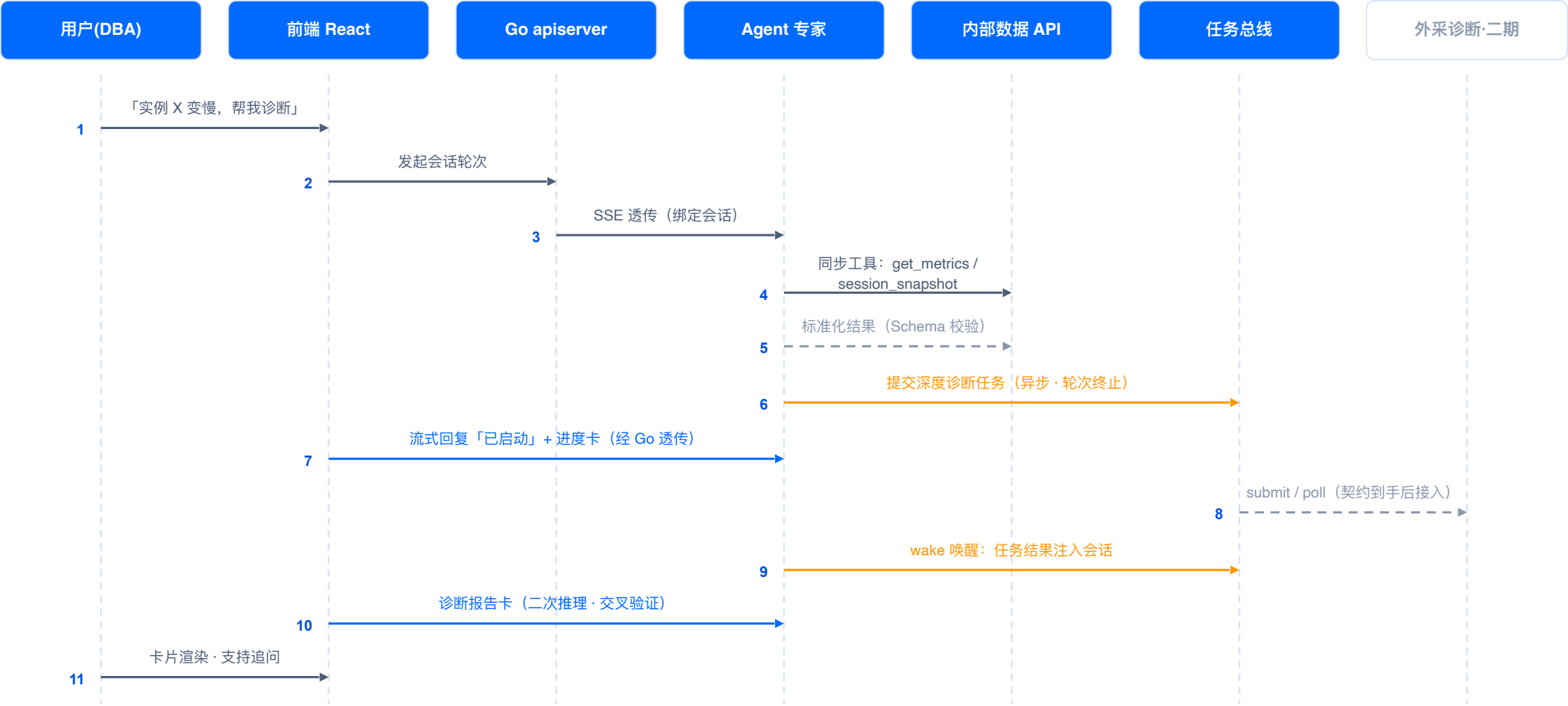
存储收口

全部表归 Go 持有，Python 无状态

防腐收口

全部旧系统调用在 Go 适配层

核心流程：智能诊断全链路时序



任务完成 ≠ 直接弹结果：唤醒专家做「二次推理」，将任务结果与会话既有证据交叉验证后出结论

数据架构：三类通道 · 双数据流演进

a · 本地沉淀通道

- 元数据 / 告警 / Issue
- 定时拉取 + 同步水位（幂等）
- 告警统一 Issue 化（平台内流转）

b · 实时直连通道

- 会话 / 锁 / 慢SQL / 执行计划
- 实例接入网关：只读账号
- 操作审计 · 超时熔断（不缓存）

c · 旧系统 API 通道

- 时序指标经查询代理（带缓存）
- 限流 / 排队 / 降级返最近缓存值
- 二期被自建采集替换，上层无感



诊断档案 = 一等公民资产

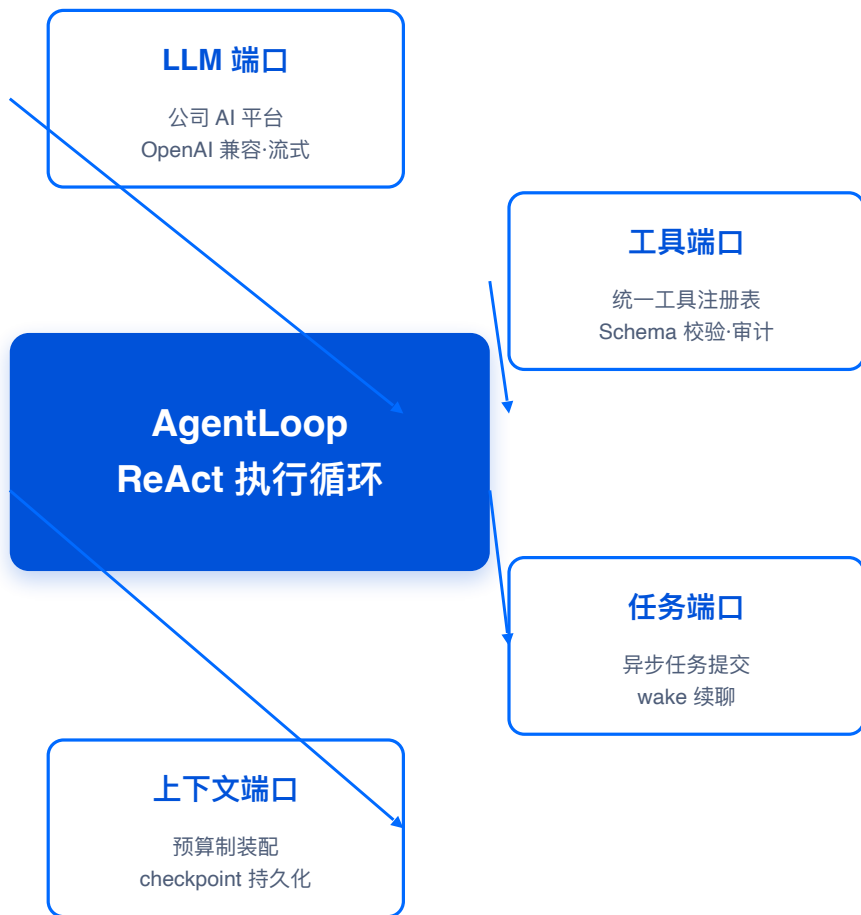
- 会话 / 轮次 / 工具轨迹 / 报告全量落库
- 任一会话可完整回放（故障复盘 + 提示词回归）
- 大结果存 JSONB 引用，控制库体积



指标归一化 · 最小白名单策略（MVP）

- 只归一大盘与诊断所需 20~30 个核心指标
- 映射表落库：映射是数据不是代码，新增 = 插一行
- 未命中明确报错并给出注册指引，不猜测映射

Agent 执行框架：四端口 + ReAct 循环（LangGraph 落地）



- 一套运行时，所有专家复用
- AgentDefinition 版本化
- 有界执行

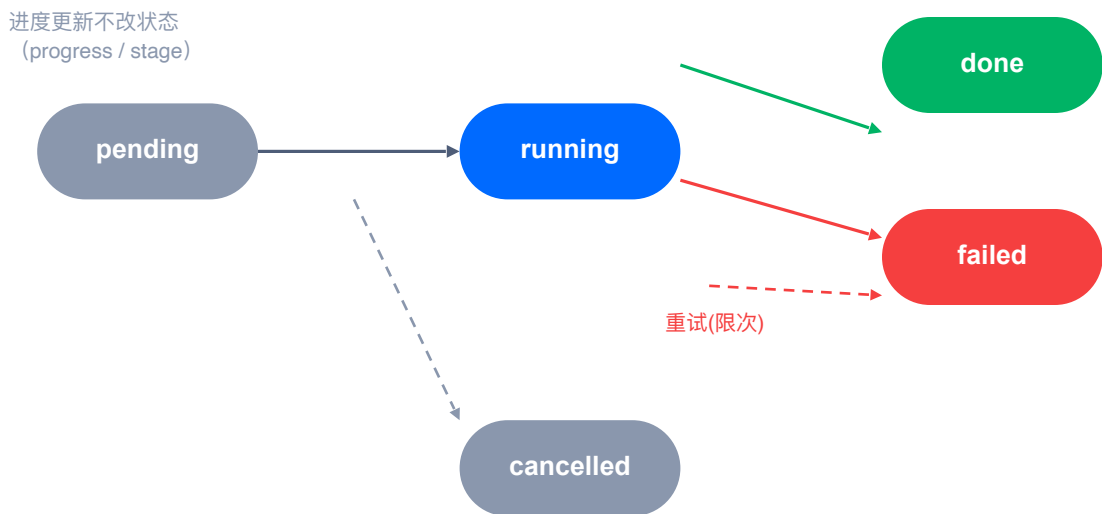
LangGraph 落地映射（Python 承载）

AgentLoop 循环	→ StateGraph: agent→tools→agent
流式输出	→ astream_events → SSE 六类事件
上下文端口	→ 自定义 Checkpointer (经 Go API 落 PG)
任务续聊	→ 异步任务 END + wake 回调续跑
预算护栏	→ recursion_limit + guard 节点

专家 = 声明式配置 (AgentDefinition)，基础设施全部挂接端口之下可替换

异步任务与续聊闭环：DB 为唯一事实源

DIAG_TASK 状态机



- 每次状态迁移写 task_event 流水（前态/后态/时间戳）
- 内存调度仅加速器：重启后 pending 重调度、running 对账改判
- call_id 全局唯一 + 工具限流：防 Agent 循环重复提交

续聊闭环（分钟级任务不阻塞对话）

- 1 专家提交异步任务**
本轮结束：回复「已启动」+ 进度卡
- 2 进度流式更新**
progress/stage 定向更新进度卡（幂等合并）
- 3 完成 wake 回调**
Go 调度器 → agentcluster 唤醒绑定专家
- 4 系统轮次注入**
任务结果作为输入创建 system turn
- 5 二次推理出报告**
与既有证据交叉验证 → 诊断报告卡

统一工具注册表：能力插件化，注册即接入

ToolDefinition 关键字段

<code>description</code>	给 LLM 看的说明书：何时用 / 不适用
<code>input/output_schema</code>	JSON Schema 双向强校验
<code>execution_mode</code>	sync / async 一等公民
<code>risk_level</code>	L0 只读自主 · L1/L2 需确认审批
<code>db_types + routing</code>	按实例库类型路由候选工具
<code>rate_limit</code>	防 Agent 循环打挂下游

外采诊断接入：三段式适配器



契约未到已落地规范与 shadow 注册位——契约到手仅实现一个 Go 适配器即接入，注册表 / 专家 / 前端零改动

MVP 自建工具清单（全部 L0 只读）

工具	类别	消费方	数据来源
<code>builtin_get_metrics</code>	metrics	问数/诊断	指标查询代理
<code>builtin_metric_anomaly</code>	metrics	诊断	阈值/环比检测
<code>builtin_list_alerts</code>	alert	问数/诊断	本地 Issue 记录
<code>builtin_list_instances</code>	topology	问数	DBaaS 同步元数据
<code>builtin_top_slow_queries</code>	slow_sql	问数/诊断	慢日志 TOP
<code>builtin_session_snapshot</code>	session	诊断	网关只读直连
<code>builtin_lock_analysis</code>	lock	诊断	锁等待链
<code>builtin_metric_deep_scan</code>	metrics·异步	诊断	验证异步闭环

问数专家仅绑定 metrics / alert / slow_sql / topology 四类（NL2Metric 白名单）

Generative UI 卡片协议：AI 输出即界面

统一信封 (Envelope)

<code>card_id</code>	全局唯一 · 流式幂等合并
<code>card_type</code>	类型枚举 (只增不改)
<code>status</code>	streaming → final (定稿只读)
<code>source</code>	溯源: 会话/轮次/Agent/工具
<code>context</code>	运维上下文: 实例 + 时间窗
<code>payload</code>	类型特化数据 (Schema 定义)
<code>interactions</code>	ask / drilldown / open_link...
<code>fallback_text</code>	降级 / 复制 / 纯文本场景

向前兼容: 未知 card_type 走 FallbackRenderer (标题 + 摘要), 绝不白屏; 版本不兼容降级为 fallback_text

MVP 五种卡片



富文本回复
text · Markdown 容器



图表卡
metric_chart · 时序 + 类目分布



数据表卡
data_table · 行级「问 AI」追问



进度卡
task_progress · 异步任务步骤条



诊断报告卡
diagnosis_report · 根因置信度/证据链



渲染器注册表 (前端): 新增卡片类型 = 注册渲染器组件, 协议与 AI 层零互改 (依赖倒置); 卡片容器统一处理信封, 业务渲染器只消费 payload

当前进展：前端 MVP 已完成并通过验收

- ✔

React 18 + TS + Vite 全新实现（约 4200 行 / 25 文件）
- ✔

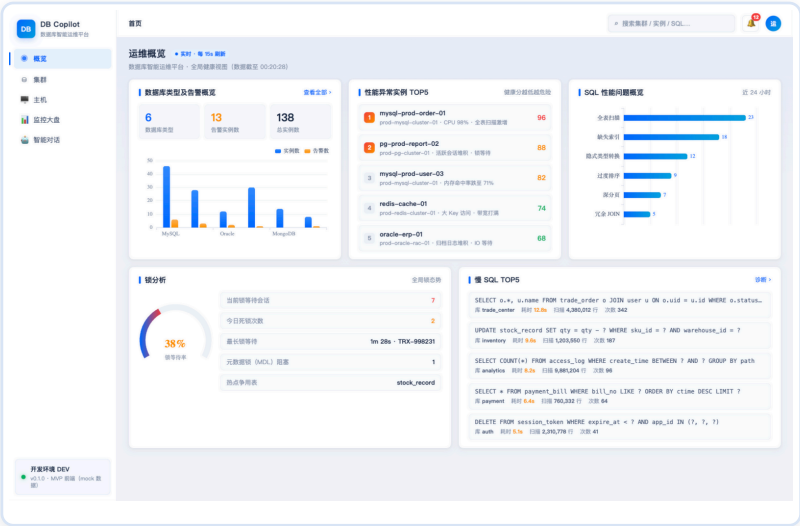
智能对话：多会话 + SSE 六类事件流式渲染
- ✔

Mock / Api 双 Provider：后端就绪换 URL，页面零改动
- ✔

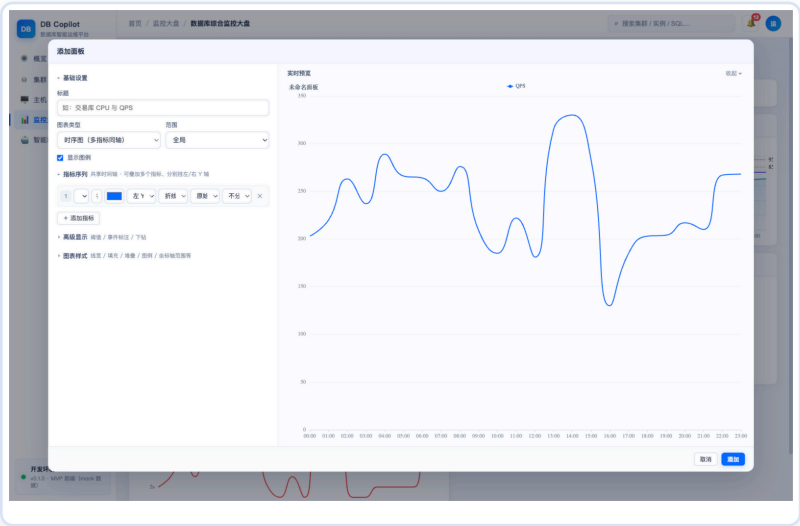
大盘完整复刻：多大盘管理 · 面板编辑器 · 实时预览
- ✔

五种卡片渲染器注册表（协议对齐，联调即验收）
- ✔

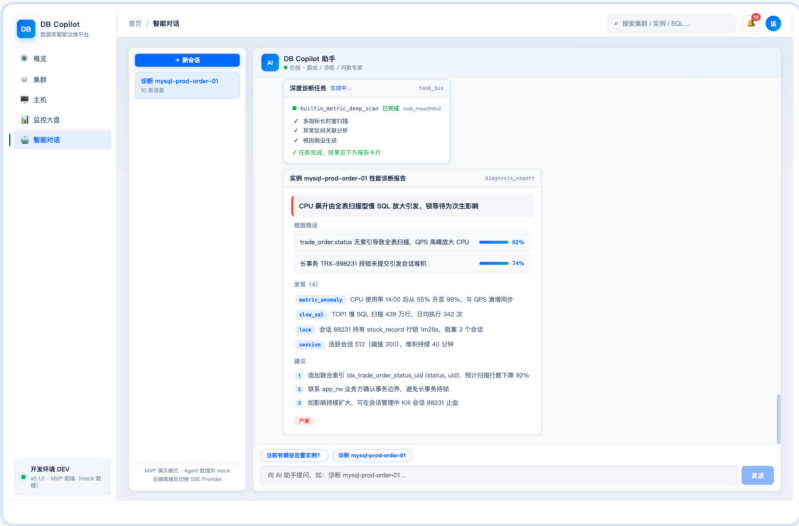
修复 echarts visualMap 缺陷（原型遗留隐藏 bug）



运维概览：类型/告警/异常 TOP5/锁分析



大面板编辑器：多指标 + 实时预览



智能诊断全流程：轨迹 + 卡片 + 报告

截图为 React 真实运行界面（mock 数据）——对 fixtures 契约开发，后端就绪后直接切换

任务分派：三个任务系列（本人 + 工程师 A + 工程师 B）



系列 1 · 前端 + apiserver

本人 · 平台入口与数据枢纽

1.1 骨架与契约桩 (W1-2)

1.2 会话与档案服务 (W3-4)

1.3 内部工具数据 API (W3-5)

1.4 任务总线调度 (W5-6)

1.5 SSE 透传对接 (W6)

1.6 元数据/告警同步编排 (W4-6)

1.7 前端切换与联调 (W7-10)

1.8 Compose 验收交付 (W11-12)

系列出口验收:
北极星全链路 + 前端零改动切换 + 一键拉起



系列 2 · Agent 集群 (Python)

工程师 A · 多专家对话服务

2.1 契约样品库 fixtures (W2-3)

2.2 Agent 运行时基座 (W3-6)

2.3 路由+诊断专家+续聊闭环 (W5-8)

2.4 问数专家 (W9-10)

系列出口验收:
fixtures 回放=真实链路 · 无状态 2 副本 50 并发



系列 3 · 数据与集成

工程师 B · 库表基座 + 防腐 + 外采

3.1 PG Schema + 部署编排 (W1-2)

3.2 旧系统适配器 x3 (W3-6)

3.3 外采契约跟进 + 适配器 (契约后 1 周)

系列出口验收:
迁移幂等 · 适配器注入即用 · 外采 shadow→active

排期总览：三线并行 · 三个集成关卡



验收机制：契约优先，集成 = 换 URL

契约优先工作法

- 1

文档契约 v1.1

内部 API / SSE 事件 / 卡片 JSON / 工具清单 / 状态机
- 2

contracts/ fixtures

JSON 样品 + Schema 校验，进 CI
- 3

三系列各自对 fixtures 自测

不依赖对方进度
- 4

集成 = 换 URL / 注入实现

与前端已验证的 Mock→Api 模式同构

纪律：契约变更必须先改 fixture 并三方会签，再改实现

验收标准示例（全部可执行 / 可量化）

- ✓ pytest 契约回放一致 · LLM 路由准确率 ≥80%
- ✓ 迁移幂等两遍无差异 · 2 副本 50 并发无串扰
- ✓ 越界问数 100% 拒答 · 重启恢复对账通过

风险与缓解

- R1

外采契约未到

适配器抽象 + 自建工具兜底 + 周会跟踪索取
- R2

网络连通性未确认

W1 确认部署位与旧系统/AI 平台网络
- R3

AI 平台模型未定名

LLM 端口配置化，任一兼容端点可开发
- R4

契约漂移

fixtures 进 CI + 三方会签纪律
- R5

系列2 关键路径

2.3 延后时本人以内部 API 桩提前联调

NORTH STAR

实例变慢 → 大盘发现异常 → 一句话发起诊断 → 真实证据采集 → 异步深度扫描 → 可追问的诊断报告

MVP 北极星场景——一次链路验证五项架构核心假设

需要支持的事项



工程师 A / B 于 W1 到岗

Python AI 方向 x1 · 数据集成方向 x1



网络连通性 W1 内确认

部署位 ↔ 旧监控 / DBaaS / AI 平台 / 外采



外采 API 契约本周索取

唯一外部硬依赖，行动项已列